VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Dharunya Balavelavan (1BM23CS090)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Dharunya Balavelavan (1BM23CS090),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

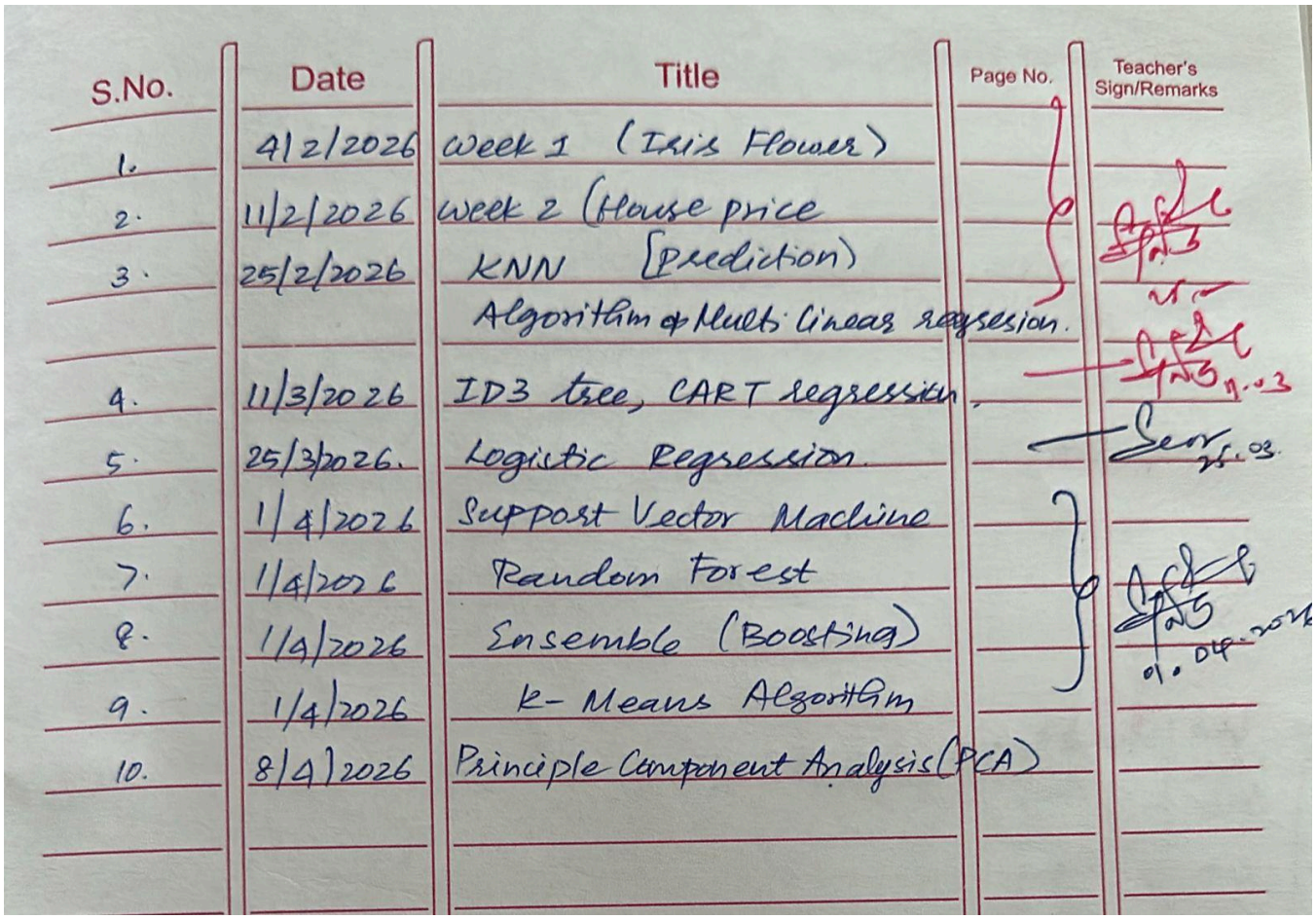
Saritha A. N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	4-2-2026	Write a python program to import and export data using Pandas library functions	
2	11-2-2026	Demonstrate various data pre-processing techniques for a given dataset	
3	25-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	
4	25-3-2026	Build Logistic Regression Model for a given dataset	
5	11-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	
6	25-2-2026	Build KNN Classification model for a given dataset.	
7	1-4-2026	Build Support vector machine model for a given dataset	
8	1-4-2026	Implement Random forest ensemble method on a given dataset.	
9	1-4-2026	Implement Boosting ensemble method on a given dataset.	
10	1-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	
11	8-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	

Github Link:

https://github.com/Dharunya21/MACHINE-LEARNING



S.No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.	4/2/2026	Week 1 (Iris Flower)		
2.	11/2/2026	week 2 (House price		
3.	25/2/2026	KNN [prediction)		
		Algorithm & Multi Linear regression.		
4.	11/3/2026	ID3 tree, CART regression.		
5.	25/3/2026.	Logistic Regression.		
6.	1/4/2026	Support Vector Machine		
7.	1/4/2026	Random Forest		
8.	1/4/2026	Ensemble (Boosting)		
9.	1/4/2026	K- Means Algorithm		
10.	8/4/2026	Principle Component Analysis (PCA)		

<u>Program 1</u>

Write a python program to import and export data using Pandas library functions

Screenshot

4/2/2026

Week-1

1). Tools:

a). Numpy: It is used for fast numerical operations and works with arrays.

b) Pandas: It is used to work with structured data (rows + column)

c) Matplotlib: It helps to visualize data using graphs and plots.

Task : 1 :

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns= iris.feature_names)
df['species'] = iris.target
df.head()
```

Index	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Columns = 5.

Task : 2

```
df.shape
```

=> (150, 5)

Features = 4.

Task : 3

df.columns

df.info()

#	column	Non-null count	Dtype
0	sepal length (cm)	150	float64
1	sepal width (cm)	150	float64
2	petal length (cm)	150	float64
3	petal width (cm)	150	float64
4	species (	150	int64

⇒ output column : species.

Task 4:

df.descsibe()

	sepal length	sepal width	petal length	petal width	species
count	150.000	150.0	150	150	150
mean	5.843	3.057	3.75	1.19	1
std	0.828	0.435	1.76	0.76	0.81
min	4.3	2.0	1.0	0.1	0
25%	5.1	2.8	1.6	0.3	0
50%	5.8	3.0	4.35	1.3	1
75%	6.4	3.3	5.1	1.8	2
max	7.9	4.4	6.9	2.5	2

⇒ feature with largest value = sepal length (≈ 7.9 cm)

Task : 5

df['species'].value_counts()

species	count
0	50
1	50
2	50

⇒ classes are balanced

Task : 6

import matplotl

plt.scatter(df['

plt.xlabel("se

plt.ylabel("

plt.show

⇒ Groups are

Task : 7

df[df['speci

df[df['spec

	sepal leng
50	7
51	6.4
52	6.9
53	5.5
54	6.5

⇒ Yes, val

specie

specie

Task : 8

df.c

s

sepal length

sepal width

petal length

petal width

species

Task : 6

```
import matplotlib.pyplot as plt
plt.scatter(df['sepal length'], df['petal length'])
plt.xlabel("sepal length")
plt.ylabel("petal length")
plt.show
```

⇒ Groups are formed.

Task : 7

```
df[df['species']==0].head()
df[df['species']==1].head()
```

	sepal length	sepal width	petal length	petal width	species
50	7	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

⇒ Yes, values do differ

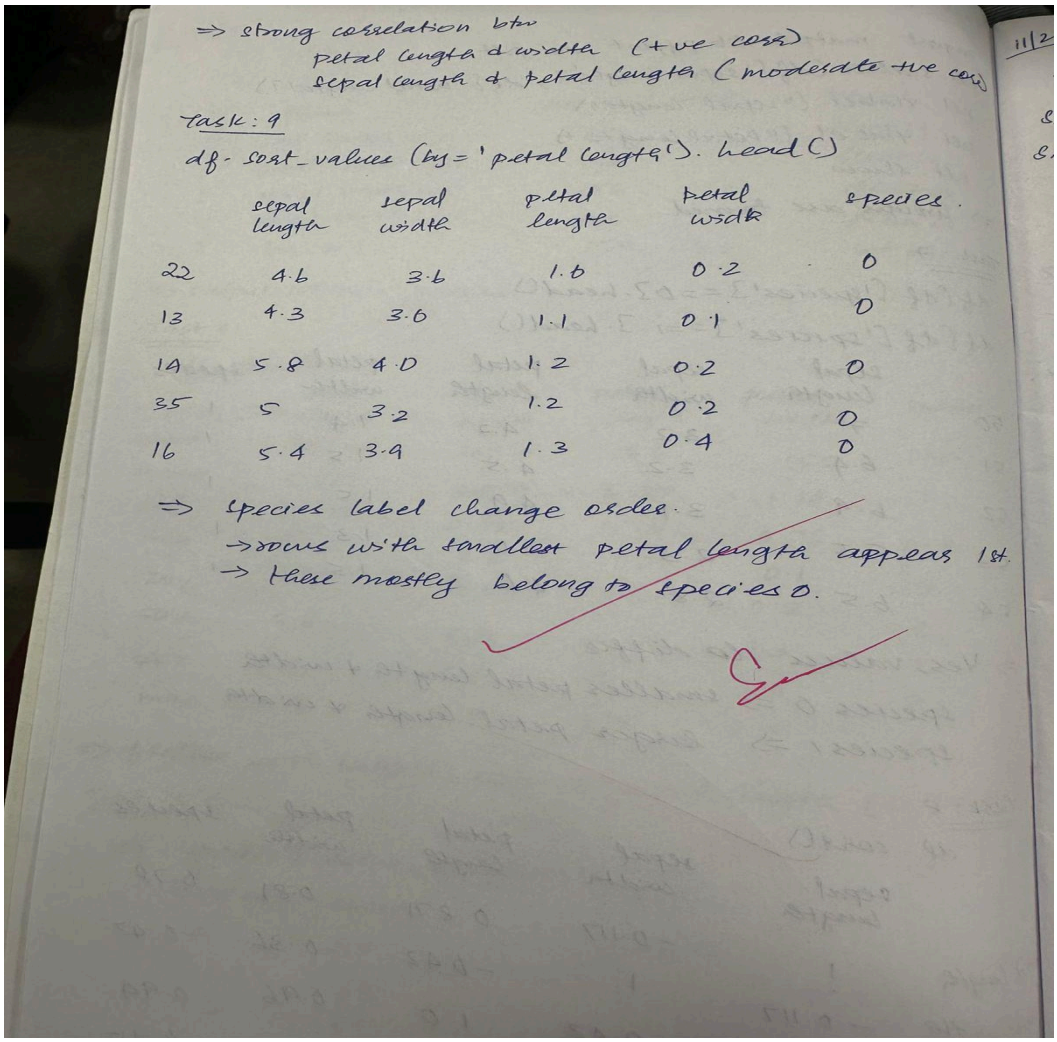
species 0 ⇒ smaller petal length & width

species 1 ⇒ larger petal length & width

Task : 8

```
df.corr()
```

	sepal length	sepal width	petal length	petal width	species
sepal length	1	−0.117	0.871	0.81	0.78
sepal width	−0.117	1	−0.42	−0.36	−0.42
petal length	0.817	−0.42	1.0	0.96	0.94
petal width	0.817	−0.36	0.96	1	0.95
species	0.782	−0.42	0.94	0.95	1



⇒ strong correlation btw
petal length & width (+ve corr)
sepal length & petal length (moderate +ve corr)

Task: 9

df.sort_values(by='petal length').head()

	sepal length	sepal width	petal length	petal width	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

⇒ species label change order.
→ rows with smallest petal length appears 1st.
→ these mostly belong to species 0.

Code:

```
from sklearn.datasets import load_iris

import pandas as pd

import matplotlib.pyplot as plt


# Load dataset

iris = load_iris()


# Create DataFrame
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Basic info
print(df.head())
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())

# Class distribution
print(df['species'].value_counts())

# Scatter plot
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()

# Filter data
print(df[df['species'] == 0].head())
print(df[df['species'] == 1].head())

# Correlation
```

```
print(df.corr())


# Sort values

print(df.sort_values(by='petal length (cm)').head())
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot

11/2/2026.

Lab-2

House Price Prediction:

step 0 : Import required libraries.

Step 1 : Look at the big pictures.

problem type : supervised learning regression.

objectives : predict mean house value based on housing features.

evaluation metrics: RMSE

step 2 : get the data

step 3 : discover the data (EDA)

step 4 : visualize the data.

4.1 : distribution of house prices.

4.2 : income v/s house price.

4.3 : correlation heatmap

4.4 : location based visualization.

step 5 : Prepare the data

5.1 : feature engineering

5.2 : train-test

5.3 : feature scaling

step 6 : select and train Baseline Model.

6.1 Linear regression (0.74495, 0.53301, 0.57656)

step 7 : select & train advanced models.

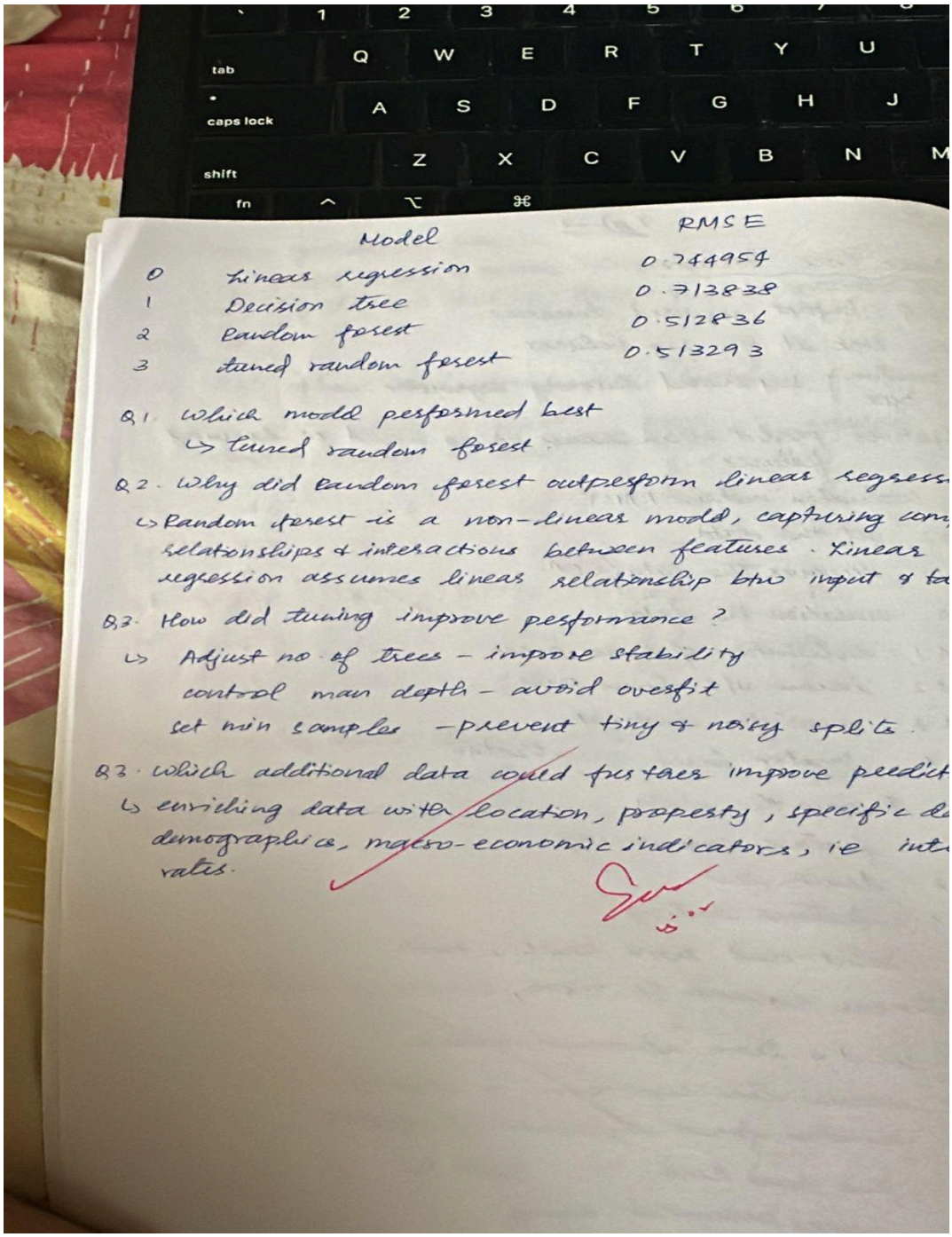
7.1 decision tree regression 0.7138773

7.2 random forest regression 0.51283 5.

Step 8 : Fine tune Model Best RM : 0.51329

8.1 hyper parameter tuning

step 9 : Model comparsion.



	Model	RMSE
0	linear regression	0.744954
1	Decision tree	0.713838
2	Random forest	0.512836
3	tuned random forest	0.513293

Q1. Which model performed best
↳ tuned random forest

Q2. Why did random forest outperform linear regress
↳ Random forest is a non-linear model, capturing com
relationships & interactions between features. Linear
regression assumes linear relationship btw input & ta

Q3. How did tuning improve performance?
↳ Adjust no of trees - improve stability
control max depth - avoid overfit
set min samples - prevent tiny & noisy splits.

Q3. Which additional data could features improve predict
↳ enriching data with location, property, specific de
demographics, macro-economic indicators, ie inte
rates.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
housing = fetch_california_housing(as_frame=True)

# Create DataFrame
df = housing.frame

# Display basic info
print(df.head())
print(df.shape)
print(df.info())

# Summary statistics
print(df.describe())
```

```
# Check missing values

print(df.isnull().sum())


# Distribution plot

plt.figure()

sns.histplot(df['MedHouseVal'], bins=30, kde=True)

plt.xlabel("Median House Value")

plt.ylabel("Frequency")

plt.title("Distribution of House Prices")

plt.show()


# Scatter plot

plt.figure()

sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)

plt.xlabel("Median Income")

plt.ylabel("Median House Value")

plt.title("Income vs House Price")

plt.show()
```

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

11/3/2026

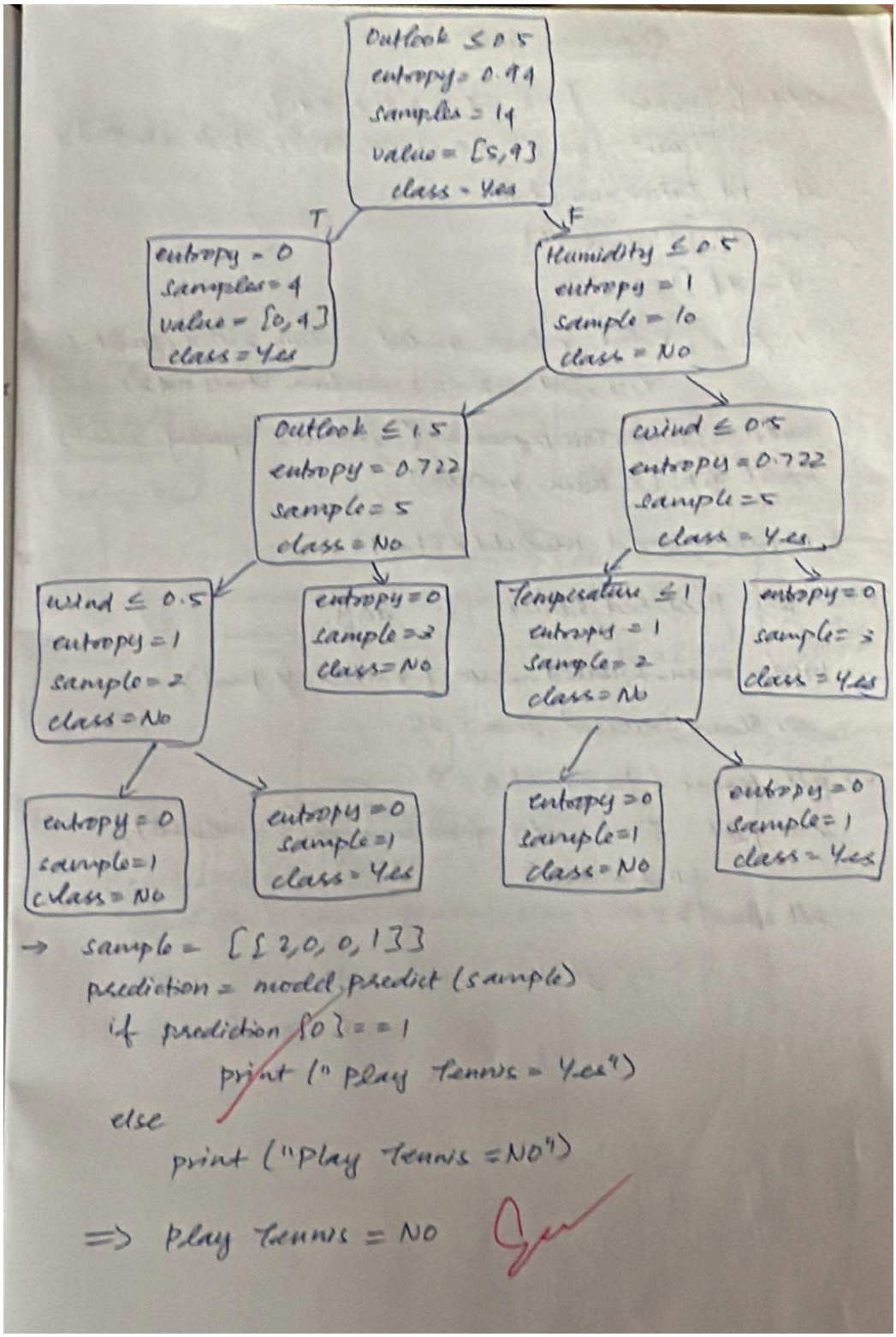
ID3 Tree

```
-> import pandas as pd
   from sklearn.tree import DecisionTreeClassifier
   from sklearn.preprocessing import LabelEncoder
   from sklearn import tree
   import matplotlib.pyplot as plt
-> df = pd.DataFrame(data)
```

=>

	Outlook	Temperature	Humidity	Wind	PlayTenn
0	Sunny	Hot	High	Weak	No
1	Overcast	Cool	Low	Strong	Yes

```
-> le = LabelEncoder()
   for column in df.columns:
       df[column] = le.fit_transform(df[column])
-> x = df.drop('PlayTennis', axis=1)
   y = df['PlayTennis']
-> model = DecisionTreeClassifier(criterion = "entropy")
   model.fit(x, y)
-> plt.figure(figsize = (12, 8))
   tree.plot_tree(model, feature_names = x.columns,
     class_names = ["No", "Yes"], filled=True)
   plt.show()
```



→ sample = [[2, 0, 0, 1]]

prediction = model.predict (sample)

if prediction [0] == 1

print ("Play Tennis = Yes")

else

print ("Play Tennis = NO")

⇒ Play Tennis = NO

Code:

import pandas as pd

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.preprocessing import LabelEncoder

from sklearn import tree

import matplotlib.pyplot as plt



data = {

    'Outlook':
['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny','Sunny','Rain','Sunny','Overcast','Ove
rcast','Rain'],

    'Temperature':
['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild','Mild','Mild','Hot','Mild'],

    'Humidity':
['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal','High','Norm
al','High'],

    'Wind':
['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Weak','Weak','Strong','Strong','Weak','S
trong'],

    'PlayTennis': ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No']

}



df = pd.DataFrame(data)

df



le = LabelEncoder()



for column in df.columns:

    df[column] = le.fit_transform(df[column])
```

```
df

X = df.drop('PlayTennis', axis=1)
y = df['PlayTennis']

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

plt.figure(figsize=(12,8))
tree.plot_tree(model, feature_names=X.columns, class_names=["No","Yes"], filled=True)
plt.show()

sample = [[2,0,0,1]]   # example encoded values
prediction = model.predict(sample)

if prediction[0] == 1:
    print("Play Tennis: YES")
else:
    print("Play Tennis: NO")
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot

Multi-Linear Regression.

Algorithm:

1. Collect data - gather features $x_1, x_2 \ldots x_n$ & target y.
2. Preprocess data
3. Define model - $y = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$
4. Choose cost function - usually mean squared error.
5. Optimize coeffxients - using normal equation.
6. Train model - fit the model & find β values.
7. Predict - use $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$
8. evaluate - check accuracy R^2, MSE, RMSE.

Output:

selected features: [bmi, bp, s1]

RMSE : 61.08

MAE : 49.99

R2 score : 0.29

coefficients : [39.53 18.96 0.46]

Intercept : 153.73

Code:

```
import numpy as np

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# Load dataset

data = load_diabetes()

X = data.data
```

```
y = data.target

# Select features
selected_features = ['bmi', 'bp', 's1']

# Get feature indices
feature_names = data.feature_names
print("All features:", feature_names)

selected_indices = [feature_names.index(f) for f in selected_features]

# Extract selected features
X_selected = X[:, selected_indices]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, random_state=42
)

# Standardization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Train Linear Regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

# Predictions
y_pred_lr = lin_reg.predict(X_test_scaled)

# Evaluation
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Selected features:", selected_features)
print("RMSE:", rmse_lr)
print("MAE:", mae_lr)
print("R2 Score:", r2_lr)
print("Coefficients:", lin_reg.coef_)
print("Intercept:", lin_reg.intercept_)
```

Program 5

Build Logistic Regression Model for a given dataset

Screenshot

25/3/2026.

Logistic Regression

```
→ df = pd.read_csv("HR_comma_sep.csv")
  print(df.head())
  print(df.info())
```

satisfaction level	last_evaluation	number_project	average_monthly_hours	time_spend_company

work_accident	left	promotion_last_5years	department

```
→ print(df['left'].value_counts())
  sns.countplot(x='left', data=df)
  plt.title("Employee Retention Count")
→ corr = df.corr(numeric_only=True)
  plt.figure(figsize=(10,6))
  sns.heatmap(corr, annot=True, cmap='coolwarm')
  plt.title("Correlation Heatmap")
→ salary_retention = df.groupby('salary')['left'].mean()
  salary_retention.plot(kind='bar')
  plt.ylabel("Probability of leaving")
→ dept_retention = df.groupby('Department')['left'].mean()
  dept_retention.plot(kind='bar', figsize=(10,5))
  plt.ylabel("Probability of leaving")
  plt.xticks(rotation=45)
→ df = pd.get_dummies(df, columns=['salary', 'Department'],
        drop_first=True)
→ x = df.drop('left', axis=1)
  y = df['left']
→ x_train, x_test, y_train, y_test = train_test_split(x, y,
        test_size=0.2, random_state=42)
→ y_pred = model.predict(x_test)
```

→ accuracy = accuracy_score(y_test, y_pred)
print (confusion_matrix (y_test, y_pred)
print (classification_report (y_test, y_pred))

⇒. Accuracy : 0.78233
Confusion Matrix : [[2117 177]
[476 2303]]

Classification report :

	Precision	recall	f1-score	support.
0	0.82	0.92	0.87	2294
1	0.57	0.33	0.41	706
accuracy.			0.78	3000
macro avg	0.69	0.62	0.64	3000
weighted avg	0.76	0.78	0.76	3000

Observation.

(i) Important variables affecting retention
* Satisfaction level
* Salary
* Average monthly hours
* Promotion
* time spent in company
* Department

(ii) Model Accuracy (80%)

↳ Yes, it is good
* model predicts most cases correctly
* suitable for a basic model.

25.03.2026

Code:

from google.colab import files

```
uploaded=files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("HR_comma_sep.csv")

# Preview
print(df.head())
print(df.info())

print(df.describe())

print(df['left'].value_counts())
sns.countplot(x='left', data=df)
plt.title("Employee Retention Count (0 = Stayed, 1 = Left)")
plt.show()

corr = df.corr(numeric_only=True)
plt.figure(figsize=(10,6))
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.title("Correlation Heatmap")
plt.show()

salary_retention = df.groupby('salary')['left'].mean()

salary_retention.plot(kind='bar')
plt.title("Impact of Salary on Employee Retention")
plt.ylabel("Probability of Leaving")
plt.show()

dept_retention = df.groupby('Department')['left'].mean()

dept_retention.plot(kind='bar', figsize=(10,5))
plt.title("Department vs Employee Retention")
plt.ylabel("Probability of Leaving")
plt.xticks(rotation=45)
plt.show()

# One-hot encoding
df = pd.get_dummies(df, columns=['salary', 'Department'], drop_first=True)

X = df.drop('left', axis=1)
y = df['left']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Program 6

Build KNN Classification model for a given dataset.

Screenshot

5

28/02/2026

Lab-3

KNN (K-Nearest Neighbours) Algorithm:

* Choose a value for k:
  we first choose a value for K. This determines the no. of nearest neighbours used to make prediction.
* Calculate the distance:
  After choosing k, we calculate the distance between each data point in the training set and the target data point for which a prediction is being made.
* Find the k nearest neighbour:
  After calculating the distance between the existing data points and the new data point, we identify K nearest neighbours by selecting the K data points nearest to the new data point.
* Calculate the prediction:
  After finding the neighbours, we calculate the value of the dependent variable for the new data point. Then, we take the average of the target values of the K nearest neighbours.

Application:

California Housing Prices

```
=> data = fetch_california_housing()
   x = data.data
   y = data.target

=> x_train, x_test, y_train, y_test = train_test_split
      (x, y, test_size = 0.2, random_state = 42)

=> knn = KNeighboursRegressor (n_neighbours = 3)
   knn.fit (x_train, y_train)
```

Code:

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor

# Load dataset
data = fetch_california_housing()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train KNN Regressor
knn = KNeighborsRegressor(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
# New data prediction
new_house = np.array([[8.3252, 41, 6.984, 1.023,
                       322, 2.555, 37.88, -122.23]])

new_house_scaled = scaler.transform(new_house)

predicted_price = knn.predict(new_house_scaled)

print("Predicted House Price (in $100,000s):", predicted_price[0])
print("Predicted Price in Dollars: $", predicted_price[0] * 100000)
```

Program 7

Build Support vector machine model for a given dataset.

Screenshot

11/9/2026.

Support Vector Machine

```
x = np.array([[1,60,50],[2,65,55] ... 3])
y = np.array([0,0,0,1,1,1,1,1,1,0,0,0,1,1,1,1,1,1])
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
model = SVC(kernel='linear')
model.fit(x_train, y_train)
```

```
=>  SVC
    SVC(kernel='linear')
```

```
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification report:", classification_report
    (y_test, y_pred))
```

=> Accuracy = 1.0

Classification report:

	precision	recall	f1-score	support
0	1	1	1	2
1	1	1	1	2

```
new_student = np.array([[5,70,65]])
new_student = scaler.transform(new_student)
prediction = model.predict(new_student)
if prediction[0] == 1:
    print("Result: Pass")
else:
    print("Result: fail")
```

=> Result: pass.

Code:

```
import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report


X = np.array([

    [1, 60, 50],

    [2, 65, 55],

    [3, 70, 60],

    [4, 75, 65],

    [5, 80, 70],

    [6, 85, 75],

    [7, 90, 80],

    [8, 95, 85],

    [2, 50, 45],

    [3, 55, 50],

    [4, 60, 55],

    [5, 65, 60],

    [6, 70, 65],

    [7, 75, 70],

    [8, 80, 75],
```

```
    [9, 85, 80]
])

y = np.array([
    0,0,0,1,1,1,1,1,
    0,0,0,1,1,1,1,1
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='linear')   # try 'rbf' also
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
new_student = np.array([[5, 70, 65]])  # example input
new_student = scaler.transform(new_student)


prediction = model.predict(new_student)


if prediction[0] == 1:
    print("\nResult: Pass")
else:
    print("\nResult: Fail")
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

Random forest

```
df = pd.read_csv("random_forest_sample.csv")
df.head()
x = df.iloc[:, :-1]
y = df.iloc[:, -1]
x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100,
        random_state=42)
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification report:", classification_report
        (y_test, y_pred))
print("Confusion Matrix:", confusion_matrix(y_test,
        y_pred))
```

=> Accuracy: 1.0

classification Report:

	precision	recall	f1-score	support
0	1	1	1	3
1	1	1	1	1

Confusion Matrix:

```
[[3 0]
 [0 1]]
```

Code:

```
import numpy as np

import pandas as pd
```

```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Load Iris dataset (optional reference)

from sklearn.datasets import load_iris

data = load_iris()

X = data.data

y = data.target


print("Feature shape:", X.shape)

print("Target shape:", y.shape)


# Create sample dataset

data = {

    "feature1": [5.1,4.9,4.7,4.6,5.0,5.4,6.4,6.9,5.5,6.5,5.7,6.3,6.5,7.6,4.9,7.3,6.7,7.2],

    "feature2": [3.5,3.0,3.2,3.1,3.6,3.9,3.2,3.1,2.3,2.8,2.8,3.3,3.0,3.0,2.5,2.9,2.5,3.6],

    "feature3": [1.4,1.4,1.3,1.5,1.4,1.7,4.5,4.9,4.0,4.6,4.5,4.7,5.8,6.6,4.5,6.3,5.8,6.1],

    "feature4": [0.2,0.2,0.2,0.2,0.2,0.4,1.5,1.5,1.3,1.5,1.3,1.6,2.2,2.1,1.7,1.8,1.8,2.5],

    "target":   [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]

}


df = pd.DataFrame(data)
```

```
# Save CSV
file_name = "random_forest_sample.csv"
df.to_csv(file_name, index=False)

# (Colab) Download file
from google.colab import files
files.download(file_name)

# (Colab) Upload file
uploaded = files.upload()

# Load dataset
df = pd.read_csv("random_forest_sample.csv")

print(df.head())
print(df.info())

# Split features and target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)

# Train Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Boosting ensemble method.

```
data = load_breast_cancer (as_frame = True)
df = data.frame
df.head()
x = df.drop('target', axis=1)
y = df['target']
x_train, x_test, y_train, y_test = train_test_split (
        x, y, test_size=0.2, random_state=42)
model = AdaBoostClassifier (n_estimators = 50,
                random_state =42)
model.fit (x_train, y_train)
y_pred = model.predict (x_test)
accuracy = accuracy_score (y_test, y_pred)
print ("Accuracy")
cm = confusion_matrix (y_test, y_pred)
```

=> Accuracy = 0.9649

```
[[40, 3]
 [1, 70]]
```

Code:

import pandas as pd

import numpy as np

from sklearn.datasets import load_breast_cancer

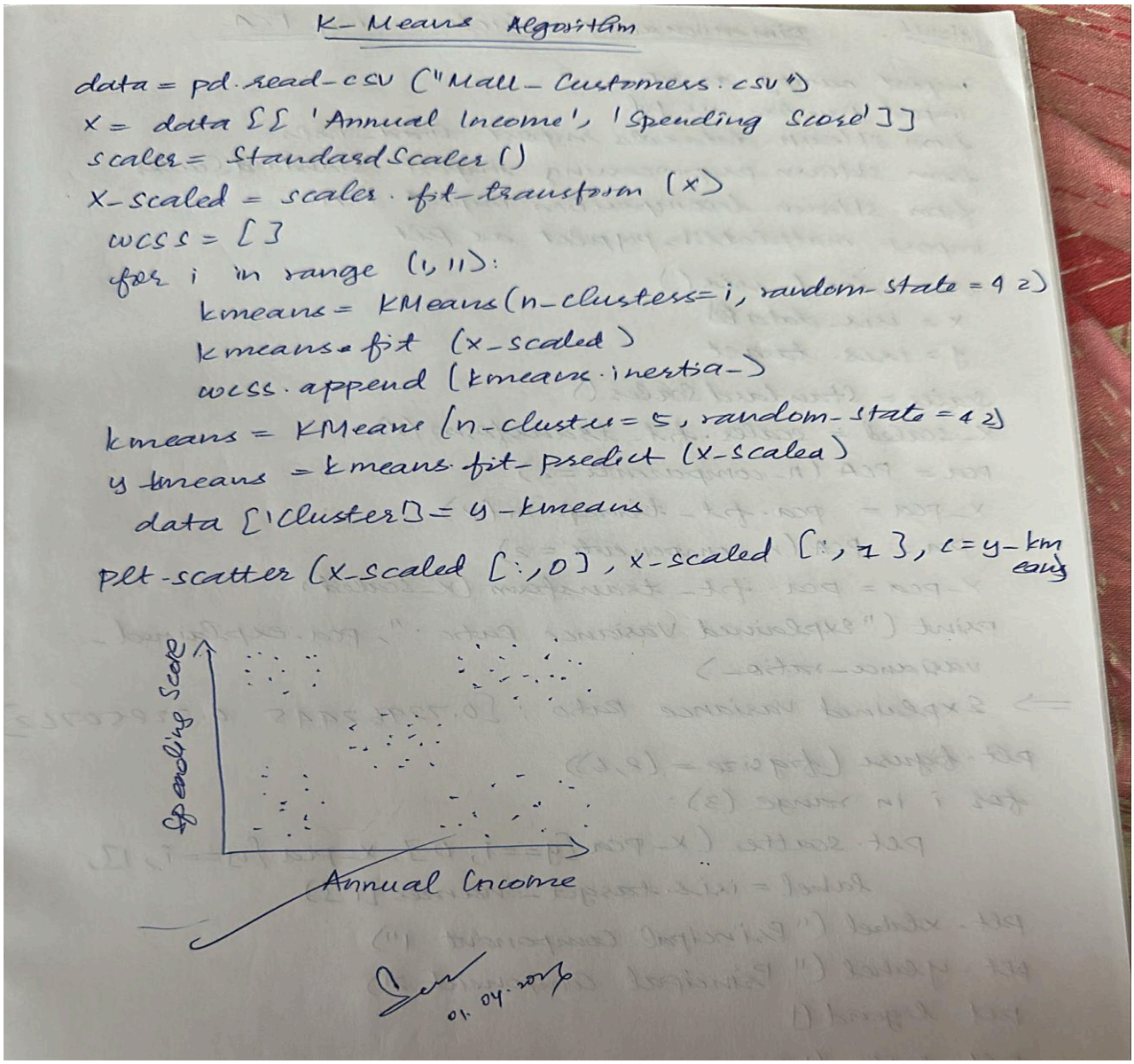
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
data = load_breast_cancer(as_frame=True)
df = data.frame

# Features and target
X = df.drop('target', axis=1)
y = df['target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train AdaBoost model
model = AdaBoostClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)
```

```
# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



K-Means Algorithm

```
data = pd.read_csv ("Mall_Customers.csv")
x = data [[ 'Annual Income', 'Spending Score']]
scaler = StandardScaler()
x_scaled = scaler.fit_transform (x)
wcss = [ ]
for i in range (1, 11):
    kmeans = KMeans(n_clusters= i, random_state = 42)
    kmeans.fit (x_scaled)
    wcss.append (kmeans.inertia_)
kmeans = KMeans (n_cluster= 5, random_state = 42)
y_kmeans = kmeans.fit_predict (x_scaled)
data ['Cluster'] = y_kmeans
plt.scatter (x_scaled [:,0], x_scaled [:, 1], c=y_kmeans
```


01.04.2026

Code:

```
from google.colab import files

uploaded = files.upload()


import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("Mall_Customers.csv")
print(data.head())

# Select features
X = data[['Annual Income (k$)', 'Spending Score (1-100)']]

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method (WCSS)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
```

```
plt.ylabel("WCSS")
plt.show()

# Apply KMeans (k=5)
kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)

# Add cluster labels
data['Cluster'] = y_kmeans
print(data.head())

# Plot clusters
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans, cmap='rainbow')
plt.title("Customer Segments")
plt.xlabel("Annual Income (scaled)")
plt.ylabel("Spending Score (scaled)")
plt.show()
```

Program 11

Implement Dimensionality reduction using the Principle Component Analysis (PCA) method.

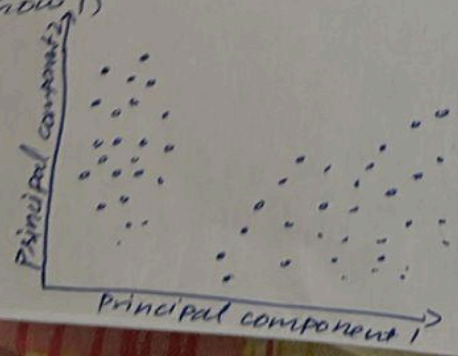
Screenshot

5/4/2026. **Dimensionality reduction using PCA.**

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA.
import matplotlib.pyplot as plt

iris = load_iris()
x = iris.data
y = iris.target
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
print("Explained Variance Ratio:", pca.explained_
variance_ratio_)
```

⟹ Explained Variance Ratio : [0.72962445 0.22850762]

```
plt.figure(figsize=(8,6))
for i in range(3):
    plt.scatter(x_pca[y==i, 0], x_pca[y==i, 1],
        label = iris.target_names[i])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.title("PCA on Iris Dataset")
plt.show()
```



Code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data
y = iris.target
# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Apply PCA (2 components)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Explained variance
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
print("Total Variance Retained:", sum(pca.explained_variance_ratio_))

# PCA components (feature contributions)
print("PCA Components:\n", pca.components_)
```

```
# Plot
plt.figure(figsize=(8, 6))
for i in range(3):
    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], label=iris.target_names[i])


plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.title("PCA on Iris Dataset")
plt.show()
```